

Muhammad Omar Zahid

L1F22BSCS0028

PDC

Section H4

Assignment 4

Q1

Question 1

a)

Sol: One to All Broadcast :

0000 $\rightarrow$ 0001] Iteration 1

0000 $\rightarrow$ 0010
0001 $\rightarrow$ 0011] Iteration 2

0000 $\rightarrow$ 0100
0001 $\rightarrow$ 0101
0010 $\rightarrow$ 0110
0011 $\rightarrow$ 0111] Iteration 3

0100 $\rightarrow$ 1100
0101 $\rightarrow$ 1101
0110 $\rightarrow$ 1110
0111 $\rightarrow$ 1111] Iteration 4

Minimum 4 iterations in All to All broadcast

0000 $\rightarrow$ a	0001 $\rightarrow$ b	0010 $\rightarrow$ c	0011 $\rightarrow$ d	0101 $\rightarrow$ f
0110 $\rightarrow$ g	0111 $\rightarrow$ h	1000 $\rightarrow$ i	1001 $\rightarrow$ j	1010 $\rightarrow$ k
1011 $\rightarrow$ l	1100 $\rightarrow$ m	1101 $\rightarrow$ n	1110 $\rightarrow$ o	1111 $\rightarrow$ p

b)

Sol: 1- One to All Scatter:

Iter 1: 0000 sends upper half of its block ['i', ... 'p'] to 1000

Iter 2: 0000 sends ['e', ... 'h'] to 0110; 1000 sends ['m', ... 'p'] to 1100

Iter 3: Nodes toggle 2nd bit, sending blocks of size 2 to their neighbour

Iter 4: Nodes toggle 1 bit send 1 final element to immediate neighbours

2- One to All Broadcast:

Node 0000 sends its element 'a' 4 iterations detailed in part a. All will contain 'a'.

3- All to All Broadcast:

Every node starts with its own letter. Over 4 iterations, nodes swap data among dimensions 1 then 2, 3, 4. All nodes have a collection of 16 letters

4- Gathering at node 0:

It is reverse of scatter. Nodes send their individual letter back until node 0000 holds full reconstructed array A.

5- Operation Sum:

Assuming the letters are mapped to numerical values, nodes pair up along dimensions starting from 4 to 1. After 4 steps, Node 0000 contains the global total sum.

Q2

Code:

```
from mpi4py import MPI
import numpy as np

def main():
    comm = MPI.COMM_WORLD

    rank = comm.Get_rank()

    size = comm.Get_size()

    if size != 16:
        if rank == 0:
            print(f"Error: This program must be run with exactly 16 processes. Current size:
{size}")

        return

    binary_id = format(rank, '04b')

    broadcast_data = None

    if rank == 0:
        broadcast_data = "Secret Threat-Intel Key"

    comm.Barrier()

    broadcast_result = comm.bcast(broadcast_data, root=0)

    if rank in [0, 1, 2, 4, 8, 15]:
        print(f"[Node {binary_id} (Rank {rank:02d})] Received Broadcast Data:
'{broadcast_result}'")

    comm.Barrier()

    if rank == 0:
        print("-" * 70)

    my_character = chr(97 + rank)

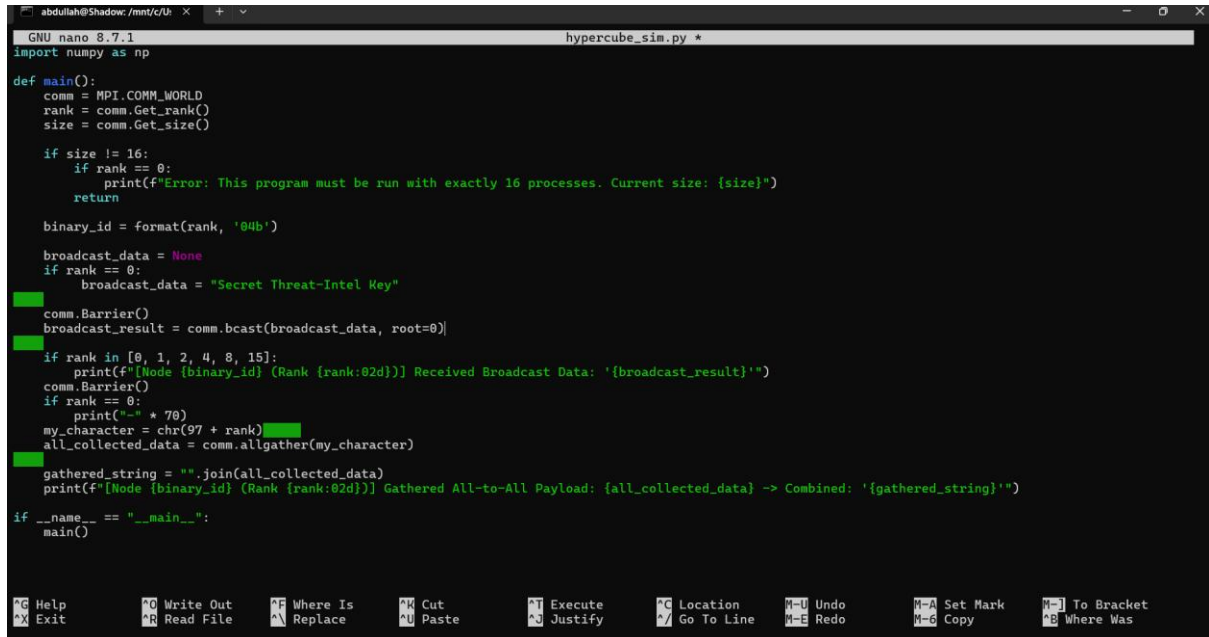
    all_collected_data = comm.allgather(my_character)
```

```
gathered_string = "".join(all_collected_data)
```

```
print(f"[Node {binary_id} (Rank {rank:02d})] Gathered All-to-All Payload:  
{all_collected_data} -> Combined: '{gathered_string}'")
```

```
if __name__ == "__main__":
```

```
    main()
```



```
GNU nano 8.7.1 hypercube_sim.py *
import numpy as np

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size != 16:
        if rank == 0:
            print(f"Error: This program must be run with exactly 16 processes. Current size: {size}")
            return

    binary_id = format(rank, '04b')

    broadcast_data = None
    if rank == 0:
        broadcast_data = "Secret Threat-Intel Key"

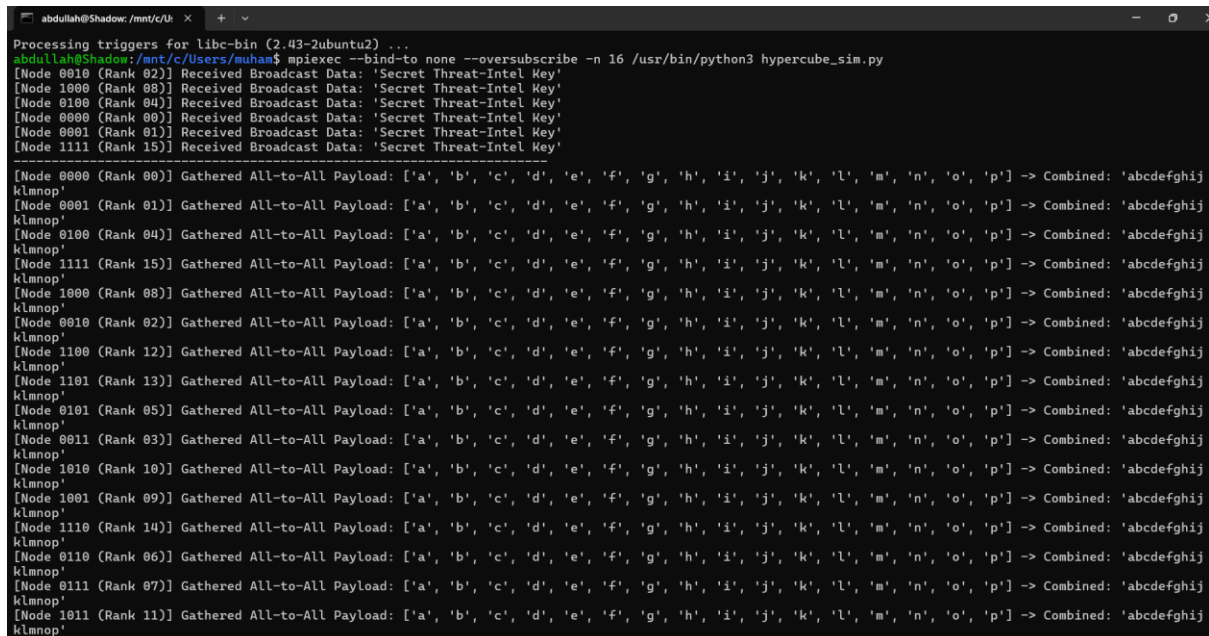
    comm.Barrier()
    broadcast_result = comm.bcast(broadcast_data, root=0)

    if rank in [0, 1, 2, 4, 8, 15]:
        print(f"[Node {binary_id} (Rank {rank:02d})] Received Broadcast Data: '{broadcast_result}'")
    comm.Barrier()
    if rank == 0:
        print("-" * 70)
    my_character = chr(97 + rank)
    all_collected_data = comm.allgather(my_character)

    gathered_string = "".join(all_collected_data)
    print(f"[Node {binary_id} (Rank {rank:02d})] Gathered All-to-All Payload: {all_collected_data} -> Combined: '{gathered_string}'")

if __name__ == "__main__":
    main()
```

Output:



```
Processing triggers for libc-bin (2.43-2ubuntu2) ...
abdullah@Shadow: /mnt/c/Ab - +
abdullah@Shadow:/mnt/c/Users/muham$ mpiexec --bind-to none --oversubscribe -n 16 /usr/bin/python3 hypercube_sim.py
[Node 0010 (Rank 02)] Received Broadcast Data: 'Secret Threat-Intel Key'
[Node 1000 (Rank 08)] Received Broadcast Data: 'Secret Threat-Intel Key'
[Node 0100 (Rank 04)] Received Broadcast Data: 'Secret Threat-Intel Key'
[Node 0000 (Rank 00)] Received Broadcast Data: 'Secret Threat-Intel Key'
[Node 0001 (Rank 01)] Received Broadcast Data: 'Secret Threat-Intel Key'
[Node 1111 (Rank 15)] Received Broadcast Data: 'Secret Threat-Intel Key'
-----
[Node 0000 (Rank 00)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0001 (Rank 01)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0100 (Rank 04)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1111 (Rank 15)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1000 (Rank 08)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0010 (Rank 02)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1100 (Rank 12)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1101 (Rank 13)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0101 (Rank 05)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0011 (Rank 03)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1010 (Rank 06)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1001 (Rank 09)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1110 (Rank 14)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0110 (Rank 06)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 0111 (Rank 07)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
[Node 1011 (Rank 11)] Gathered All-to-All Payload: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p'] -> Combined: 'abcdefghijklmnp'
```